

# Entropy

Throughout,  $\log = \log_2$ , the unit is the **bit**, and  $0 \log 0 = 0$ . The first half of the lecture uses a single running example, sending instructions to a robot when every bit costs money; the second half uses the two-dice table of the Conditional Expectation lecture.

1. **Codes** Prefix-free codes are binary trees; a codeword of length  $\ell$  consumes  $2^{-\ell}$  of the tree (Kraft).
2. **Information**  $-\log p$  The ideal codeword length: the number of halvings from 1 down to  $p$ .
3. **Entropy**  $H$  Expected information; the area under stacked bars.
4. **Source coding theorem**  $H \leq \text{best average code length} < H + 1$ .
5. **Cross-entropy, relative entropy** The cost of coding for the wrong distribution, and the waste. Gibbs' inequality.
6. **Why the loss is a logarithm** An honest penalty must be  $-\log q$ ; cross-entropy loss; prediction and compression are equivalent.
7. **Wordle** A guess is a partition; its entropy is its expected information.
8. **Joint, conditional, mutual** Row entropies of the two-dice table; chain rule;  $I(X; Y)$ .
9. **Models of text** Markov chains, entropy rate, English at about 1 bit per letter, perplexity, ciphers.

## Motivation: A Robot on a Distant Moon

A rover receives a stream of instructions, each one of up, down, left, right, independent of the others, with probabilities

$$\mathbb{P}(\text{up}) = \frac{1}{2}, \quad \mathbb{P}(\text{down}) = \frac{1}{4}, \quad \mathbb{P}(\text{left}) = \frac{1}{8}, \quad \mathbb{P}(\text{right}) = \frac{1}{8}.$$

Every bit sent costs money.

### A fixed-length code

up  $\mapsto$  00, down  $\mapsto$  01, left  $\mapsto$  10, right  $\mapsto$  11. The cost is 2 bits per instruction, regardless of the probabilities.

### Questions we cannot yet answer

1. Can a cleverer encoding average fewer than 2 bits per instruction?
2. If so, how low can the average go? Is there a floor, and what is it?
3. The rover's mission changes and right becomes the common instruction, but the code installed on board stays. What does that cost?

Pre-lecture viewing: "Reinventing Entropy" (3Blue1Brown, 32 min),

<https://www.youtube.com/watch?v=l6DKRf-fAAM>

# The Setting: A Source of Symbols

Fix a probability space  $(\Omega, \mathcal{F}, \mathbb{P})$ ; events are the elements of  $\mathcal{F}$ .

- $S$  is a finite set of **symbols**,  $|S| = n$ . Label them  $1, \dots, n$  if convenient, so that  $X: \Omega \rightarrow S$  is a random variable in the sense of the Refresher:  $\{X = x\} \in \mathcal{F}$  for every  $x \in S$ .
- Shorthand, used throughout:  $\{X \in A\} := \{\omega \in \Omega : X(\omega) \in A\}$  and  $\mathbb{P}(X \in A) := \mathbb{P}(\{X \in A\})$ .
- The **pmf** of  $X$  is  $p_X: S \rightarrow [0, 1]$ ,  $p_X(x) = \mathbb{P}(X = x)$ ; its **support** is  $S_X = \{x \in S : p_X(x) > 0\}$ .

## Definition (source)

A **source** is a sequence of random variables  $X_1, X_2, \dots: \Omega \rightarrow S$ , independent and all with pmf  $p_X$ . Independence, written  $X_i \perp\!\!\!\perp X_j$ , means  $\mathbb{P}(X_i \in A, X_j \in B) = \mathbb{P}(X_i \in A)\mathbb{P}(X_j \in B)$  for all  $A, B \subseteq S$ ; for the entire sequence,

$$\mathbb{P}(X_1 = x_1, \dots, X_m = x_m) = p_X(x_1) \cdots p_X(x_m) \quad \text{for every } m \text{ and all } x_1, \dots, x_m \in S.$$

*A source is a die with  $n$  faces, rolled again and again; we are charged for reporting the outcomes.*

- The rover:  $S = \{\text{up}, \text{down}, \text{left}, \text{right}\}$ ,  $p_X = (\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \frac{1}{8})$ ,  $S_X = S$ , and  $X_i$  is the  $i$ -th instruction.

## Binary Code

Let  $\{0, 1\}^+$  be the set of non-empty finite strings of bits: 0, 1, 00, 01, ...

### Definition

A **binary code** for  $S$  is an injective function  $c: S \rightarrow \{0, 1\}^+$ ;  $c(x)$  is the **codeword** of  $x$ . Its **length function** is  $\ell_c: S \rightarrow \{1, 2, 3, \dots\}$ ,  $\ell_c(x)$  = the number of bits in  $c(x)$ , and its **average length** is

$$L(c) = \mathbb{E}[\ell_c(X)] = \sum_{x \in S} p_X(x) \ell_c(x) \quad (\text{bits per symbol}).$$

The code is **prefix-free** if for  $x \neq x'$  the string  $c(x)$  is not an initial segment of  $c(x')$ .

*Each symbol is assigned a bit string; the average length is the expected number of bits per symbol; prefix-free means the receiver can tell where each codeword ends without a separator.*

### The rover, coded

$c$ : up  $\mapsto$  0, down  $\mapsto$  10, left  $\mapsto$  110, right  $\mapsto$  111. Prefix-free: no codeword begins another.

$$L(c) = \frac{1}{2} \cdot 1 + \frac{1}{4} \cdot 2 + \frac{1}{8} \cdot 3 + \frac{1}{8} \cdot 3 = 1.75 \text{ bits per instruction, against 2.}$$

The stream 0 110 100 111 is decoded left to right with no ambiguity: up, left, down, up, right.

## Prefix-Free Codes Are Binary Trees

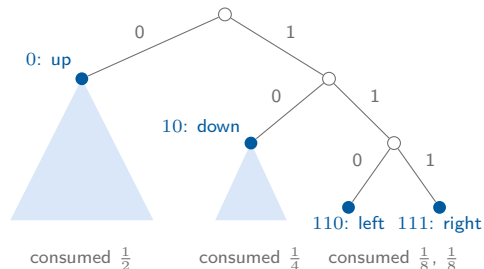
Read a codeword as a path from the root: 0 = left, 1 = right. Then  $c(x)$  is a prefix of  $c(x')$  exactly when the node of  $c(x)$  lies on the path to  $c(x')$ ; so a code is prefix-free exactly when **every codeword is a leaf**.

Choosing 0 as a codeword consumes the entire left subtree, so that half of all strings, of every length, can no longer be codewords. Choosing 10 consumes a quarter; 110 and 111 an eighth each:

$$\frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{8} = 1.$$

*A codeword of length  $\ell$  uses up the fraction  $2^{-\ell}$  of the space of codewords, and in the rover's code these fractions coincide with the probabilities.*

**Not prefix-free:** up  $\mapsto$  0, down  $\mapsto$  1, left  $\mapsto$  01, right  $\mapsto$  10. This map is injective and averages 1.5 bits, but it cannot be decoded, since 01 reads as updown or as left. Here 0 and 1 are not leaves.



|                  |                   |     |     |
|------------------|-------------------|-----|-----|
| 0: $\frac{1}{2}$ | 10: $\frac{1}{4}$ | 110 | 111 |
|------------------|-------------------|-----|-----|

the space of codewords, total 1

# Kraft's Inequality

## Theorem (Kraft, 1949)

If a prefix-free code for  $S = \{x_1, \dots, x_n\}$  has lengths  $\ell_1, \dots, \ell_n$ , then

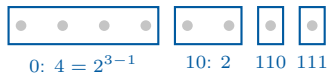
$$\sum_{i=1}^n 2^{-\ell_i} \leq 1.$$

Conversely, for any positive integers  $\ell_1, \dots, \ell_n$  with  $\sum_i 2^{-\ell_i} \leq 1$  there is a prefix-free code with exactly these lengths.

*The fractions of codeword space consumed by the codewords must fit inside 1; and any lengths that fit can be realised.*

- Rover:  $2^{-1} + 2^{-2} + 2^{-3} + 2^{-3} = 1$ . The tree is full.
- Lengths (1, 2, 2, 3) are impossible for any four symbols:  $\frac{1}{2} + \frac{1}{4} + \frac{1}{4} + \frac{1}{8} = \frac{9}{8} > 1$ .
- Lengths (2, 2, 2, 2):  $4 \cdot \frac{1}{4} = 1$ , the fixed-length code. Lengths (1, 2, 3, 4):  $\frac{15}{16} < 1$ , realisable.

the  $2^L = 8$  strings of length  $L = 3$



Beneath a codeword of length  $\ell$  lie  $2^{L-\ell}$  strings of length  $L$ ; prefix-free makes these sets disjoint.

# Kraft's Inequality: Proof

## Proof of the inequality

Let  $L = \max_i \ell_i$  and grow the tree to depth  $L$ : it has  $2^L$  leaves. Beneath the node of a codeword of length  $\ell_i$  lie  $2^{L-\ell_i}$  of them. Prefix-free means no codeword lies beneath another, so these sets of leaves are disjoint, and  $\sum_i 2^{L-\ell_i} \leq 2^L$ . Divide by  $2^L$ .  $\square$

## Proof of the converse

Order  $\ell_1 \leq \dots \leq \ell_n$  and choose codewords in turn: for  $x_i$  take the leftmost depth- $\ell_i$  node not beneath a chosen codeword. The codewords chosen so far cover the leftmost  $\sum_{j < i} 2^{-\ell_j} < 1$  of codeword space, in blocks whose sizes  $2^{-\ell_j}$  are multiples of  $2^{-\ell_i}$  (as  $\ell_j \leq \ell_i$ ); so a free depth- $\ell_i$  node exists immediately to their right. Repeat for  $i = 1, \dots, n$ .  $\square$

*Each codeword owns a disjoint block of the  $2^L$  strings of the longest length, and the blocks cannot total more than  $2^L$ .*

- The converse applied to the lengths  $(1, 2, 3, 4)$ : take 0; then 00, 01 lie beneath 0, so take 10; then 110; then 1110. Consumed  $\frac{15}{16}$ ; the node 1111 stays free.

## Information

In the rover's code, up costs 1 bit, down 2, left and right 3; and  $\frac{1}{2} = 2^{-1}$ ,  $\frac{1}{4} = 2^{-2}$ ,  $\frac{1}{8} = 2^{-3}$ . The codeword length is the number of times 1 must be halved to reach the probability.

### Definition

The **information** (surprisal) of an event is the function  $I: \{A \in \mathcal{F} : \mathbb{P}(A) > 0\} \rightarrow [0, \infty)$ ,

$$I(A) = \log \frac{1}{\mathbb{P}(A)} = -\log \mathbb{P}(A) \quad \text{bits.}$$

For a random variable  $X$  with pmf  $p_X$ , the **information function** is  $I_X: S_X \rightarrow [0, \infty)$ ,  $I_X(x) = -\log p_X(x)$ ; composing it with  $X$  gives the random variable  $I_X(X): \Omega \rightarrow [0, \infty)$ ,  $\omega \mapsto -\log p_X(X(\omega))$ .

*$I(A)$  is the number of halvings from certainty down to  $\mathbb{P}(A)$ , or equivalently the number of fair coin tosses whose outcome is as improbable as  $A$ .*

### Examples

Rover:  $I_X(\text{up}) = 1$ ,  $I_X(\text{down}) = 2$ ,  $I_X(\text{left}) = I_X(\text{right}) = 3$  bits, the codeword lengths. A six:  $\log 6 = 2.585$  bits. Double six:  $\log 36 = 5.170 = 2.585 + 2.585$  bits. Any specific sequence of 20 coin tosses: 20 bits, regardless of which sequence it is.

## Why a Logarithm: Additivity Under Independence

Wanted:  $f: (0, 1] \rightarrow [0, \infty)$ , “the information of an event of probability  $p$ ”, such that (i)  $f$  is non-increasing: a rarer event is at least as informative; (ii)  $f(pq) = f(p) + f(q)$ : for  $A \perp\!\!\!\perp B$  learning both is learning each in turn, so  $I(A \cap B) = I(A) + I(B)$  while  $\mathbb{P}(A \cap B) = \mathbb{P}(A)\mathbb{P}(B)$ ; (iii)  $f(\frac{1}{2}) = 1$ : a fair coin toss is one bit.

### Theorem

The only such function is  $f(p) = -\log p$ .

### Proof

From (ii),  $f(1) = f(1 \cdot 1) = 2f(1)$ , so  $f(1) = 0$ ; and  $f(2^{-k}) = k f(\frac{1}{2}) = k$  for  $k = 0, 1, 2, \dots$ . For rational  $r = m/n$ , applying (ii)  $n$  times gives  $n f(2^{-m/n}) = f(2^{-m}) = m$ , so  $f(2^{-r}) = r$ :  $f(p) = -\log p$  whenever  $-\log p$  is rational. For any other  $p$ , take rationals  $r < -\log p < r'$  as close as we like; then  $2^{-r'} < p < 2^{-r}$  and (i) gives  $r \leq f(p) \leq r'$ .  $\square$

*Independence multiplies probabilities while information is required to add, so  $f$  must turn products into sums, which only a logarithm does, and the convention that a coin toss is one bit fixes the base at 2.*

- With  $\ln$  in place of  $\log$  the unit is the *nat*,  $1 \text{ nat} = 1/\ln 2 = 1.443$  bits; every statement below holds with this change of unit.

## What Information Is Not

| not. . .                                          | the fact                                                                                                     | example                                                                                                                                                  |
|---------------------------------------------------|--------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| . . . about the pattern                           | $\mathbb{P}(A) = \mathbb{P}(B) \Rightarrow I(A) = I(B)$                                                      | HHHHHHHHHH and<br>HTHHTTHTHT: 10 bits each.                                                                                                              |
| . . . the length of the<br>codeword actually used | $I_X(x)$ is the <i>ideal</i> length; $\ell_c(x)$ is<br>a choice                                              | fixed-length code:<br>$\ell_c(\text{up}) = 2 \neq I_X(\text{up}) = 1$ ; rover's<br>code: $\ell_c = I_X$ , and it is optimal<br>(Section 4).              |
| . . . one object                                  | $I(A) \in [0, \infty)$ ;<br>$I_X: S_X \rightarrow [0, \infty)$ ;<br>$I_X(X): \Omega \rightarrow [0, \infty)$ | a number for an event; a function of<br>the value; a random variable, with<br>mean $H(X)$ (Section 3).                                                   |
| . . . additive without<br>independence            | $I(A \cap B) = I(A) + I(B)$ iff<br>$\mathbb{P}(A \cap B) = \mathbb{P}(A)\mathbb{P}(B)$                       | two dice, $Y$ the first, $X$ the sum:<br>$I(\{Y = 1\} \cap \{X = 2\}) = \log 36 =$<br>$5.170$ , but $I(\{Y = 1\}) + I(\{X =$<br>$2\}) = 2.585 + 5.170$ . |
| . . . negative, and zero<br>only for the certain  | $I(A) \geq 0$ , with $I(A) = 0$ iff<br>$\mathbb{P}(A) = 1$                                                   | $I(\Omega) = 0$ ; $I(\text{a six}) = 2.585$ .                                                                                                            |

*Information is determined by the probability of the event observed; the appearance of the outcome and the code used to transmit it play no part.*

## Back to the Rover: The Coded Stream Is Fair Coin Flips

Send instructions with the code 0, 10, 110, 111 and look at the stream of bits, one at a time.

### Each bit is a fair coin flip

$$\mathbb{P}(\text{1st bit} = 0) = \mathbb{P}(\text{up}) = \frac{1}{2},$$

$$\mathbb{P}(\text{2nd bit} = 0 \mid \text{1st bit} = 1) = \frac{\mathbb{P}(\text{down})}{\mathbb{P}(\text{down}) + \mathbb{P}(\text{left}) + \mathbb{P}(\text{right})} = \frac{1/4}{1/2} = \frac{1}{2},$$

$$\mathbb{P}(\text{3rd bit} = 0 \mid \text{first two} = 11) = \frac{\mathbb{P}(\text{left})}{\mathbb{P}(\text{left}) + \mathbb{P}(\text{right})} = \frac{1/8}{1/4} = \frac{1}{2}.$$

Every internal node of the tree splits the probability beneath it in half, and after a codeword ends the next instruction is independent of everything before; so every bit of the stream is 0 with probability  $\frac{1}{2}$ , independently of all earlier bits.

*An optimally coded stream has the statistics of fair coin flips, and a sequence of fair coin flips cannot be coded in fewer than one bit per flip.*

- Applying a second compression to the rover's coded stream gains nothing, since the stream has no remaining regularity to exploit.

# Entropy

The average cost of one instruction at the ideal lengths is obtained by weighting each  $I_X(x)$  by its probability.

## Definition

The **entropy** of  $X$  is the expected information of its value,

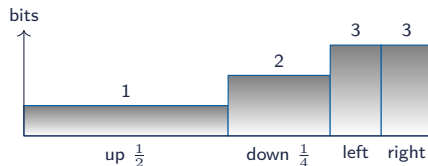
$$H(X) = \mathbb{E}[I_X(X)] = \sum_{x \in S_X} p_X(x) \log \frac{1}{p_X(x)} \quad \text{bits.}$$

It depends on  $X$  only through its pmf, so for a pmf  $p$  we also write  $H(p)$ :  $H: \Delta_n \rightarrow [0, \log n]$ ,  $\Delta_n = \{p \in [0, 1]^n : \sum_i p_i = 1\}$  (upper bound: Section 5).

*The average number of bits per symbol when every symbol gets its ideal codeword length; equivalently, the average surprise on seeing the value of  $X$ .*

## The rover

$H(\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \frac{1}{8}) = \frac{1}{2} \cdot 1 + \frac{1}{4} \cdot 2 + \frac{1}{8} \cdot 3 + \frac{1}{8} \cdot 3 = 1.75$  bits, which is the average length of the code 0, 10, 110, 111.



Bar  $i$ : width  $p_i$ , height  $\log(1/p_i)$ , area  $p_i \log(1/p_i)$ . Total area =  $H(p)$ .

## What Entropy Is Not

| not. . .                     | the fact                                                                                                                    | example                                                                                              |
|------------------------------|-----------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| . . . a random variable      | $I_X(X): \Omega \rightarrow [0, \infty)$ is the random variable;<br>$H(X) = \mathbb{E}[I_X(X)] \in [0, \infty)$ is its mean | rover: $I_X(X) \in \{1, 2, 3\}$ ;<br>$H(X) = 1.75$ .                                                 |
| . . . about the values       | $H(g(X)) = H(X)$ for injective $g$ ;<br>entropy is not variance                                                             | fair coin on $\{0, 1\}$ or on $\{0, 1000\}$ :<br>$H = 1$ bit both times; variances 0.25 and 250 000. |
| . . . negative               | $p \log \frac{1}{p} \geq 0$ , $= 0$ iff $p \in \{0, 1\}$ ; so<br>$H(X) \geq 0$ , $= 0$ iff $X$ is constant                  | $H(1, 0, 0, 0) = 0$ .                                                                                |
| . . . more than $\log n$     | $H(X) \leq \log n$ for $n$ values, $=$ iff uniform (Section 5)                                                              | fair die $\log 6 = 2.585$ ; fair coin 1.                                                             |
| . . . the number of values   | $H(X) < \log n$ unless $X$ is uniform                                                                                       | sum of two dice: 11 values,<br>$\log 11 = 3.459$ , but $H = 3.274$ (two slides on).                  |
| . . . the cost of one symbol | $I_X(x)$ is that; $H(X)$ is the average over a long stream                                                                  | rover: left costs 3 bits; the stream costs 1.75 per instruction.                                     |

*Entropy depends on the probabilities alone; the values taken do not enter.*

# Binary Entropy Function ( $h$ )

## Definition

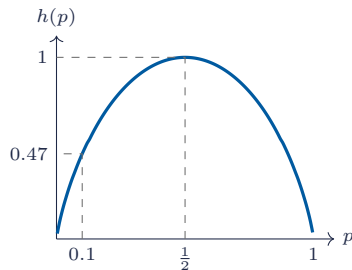
The **binary entropy function** is

$$h: [0, 1] \rightarrow [0, 1], \quad h(p) = p \log \frac{1}{p} + (1 - p) \log \frac{1}{1 - p},$$

the entropy of a coin with  $\mathbb{P}(H) = p$ , i.e. of the pmf  $(p, 1 - p)$ .

*One curve covers every two-outcome random variable, since its entropy depends only on the probability of one outcome.*

- $h(0) = h(1) = 0$ ; maximum  $h(\frac{1}{2}) = 1$ ; symmetric,  $h(p) = h(1 - p)$ ; concave, with infinite slope at 0 and 1.
- A coin showing heads 99% of the time has  $h(0.01) = 0.081$  bits, because the rare tail costs  $\log 100 = 6.64$  bits on the occasions it occurs.



Values that recur below:  $h(0.1) = 0.469$ ,  
 $h(0.2) = 0.722$ ,  $h(\frac{1}{4}) = 0.811$ ,  
 $h(0.3) = 0.881$ .

## Example: The Sum of Two Dice Costs 3.27 Bits per Roll

Two fair dice,  $Y$  the first,  $Z$  the second,  $X = Y + Z$ : the two-dice table. We compute the cost per roll of reporting the sum.

- Without the probabilities: there are 11 values, so a fixed-length code uses 4 bits, or  $\log 11 = 3.459$  if fractional lengths were allowed. Neither figure uses the fact that 7 is six times as likely as 2.
- The pmf:  $p_X(x) = n_x/36$ , where  $n_x$  is the number of cells on the anti-diagonal  $\{X = x\}$ :  
 $n_x = 1, 2, 3, 4, 5, 6, 5, 4, 3, 2, 1$  for  $x = 2, \dots, 12$ .
- For comparison: one die,  $H(Y) = \log 6 = 2.585$ ; the pair  $(Y, Z)$ , uniform on 36 cells,  $\log 36 = 5.170$ .

| $y \setminus z$ | 1 | 2 | 3 | 4  | 5  | 6  |
|-----------------|---|---|---|----|----|----|
| 1               | 2 | 3 | 4 | 5  | 6  | 7  |
| 2               | 3 | 4 | 5 | 6  | 7  | 8  |
| 3               | 4 | 5 | 6 | 7  | 8  | 9  |
| 4               | 5 | 6 | 7 | 8  | 9  | 10 |
| 5               | 6 | 7 | 8 | 9  | 10 | 11 |
| 6               | 7 | 8 | 9 | 10 | 11 | 12 |

$\{X = 7\}$ : six cells.  $\{X = 2\}$ : one cell.

$$H(X) = \sum_{x=2}^{12} \frac{n_x}{36} \log \frac{36}{n_x} = 2 \left[ \frac{1}{36} \log 36 + \frac{2}{36} \log 18 + \frac{3}{36} \log 12 + \frac{4}{36} \log 9 + \frac{5}{36} \log \frac{36}{5} \right] + \frac{6}{36} \log 6 = 3.274.$$

*The sum carries more entropy than one die and less than the pair it is computed from, since a function of the outcome cannot carry more information than the outcome (Section 8).*

# The Source Coding Theorem

Let  $L^* = \min\{L(c) : c \text{ a prefix-free code for } S\}$ , the best achievable average length.

## Theorem (Shannon, 1948)

$$H(X) \leq L^* < H(X) + 1.$$

The left inequality is an equality if and only if every  $p_X(x)$  is a power of 2, in which case the optimal code has  $\ell(x) = -\log p_X(x) = I_X(x)$  for every  $x \in S_X$ .

*Entropy is a lower bound on the average number of bits per symbol of any prefix-free code, and some prefix-free code comes within one bit of it.*

## Back to the rover

$H = 1.75$  and the code 0, 10, 110, 111 has  $L = 1.75$ : it is optimal. No prefix-free code averages fewer than 1.75 bits per instruction. That answers questions 1 and 2 of the opening slide.

- The bound  $L^* \geq H$  gives entropy its operational meaning as the least average number of bits per symbol achievable by any prefix-free code.
- The proof of the lower bound rests on an inequality that is used again for relative entropy in Section 5; the next slide states and proves it.

# Gibbs' Inequality (Lemma)

## Lemma (Gibbs' inequality, general form)

Let  $p$  be a pmf on  $S$  with support  $S_p = \{x \in S : p(x) > 0\}$  (for  $p = p_X$  this is  $S_X$ ), and let  $a: S \rightarrow [0, \infty)$  satisfy  $a > 0$  on  $S_p$  and  $\sum_{x \in S} a(x) \leq 1$ . Then

$$\sum_{x \in S_p} p(x) \log \frac{1}{a(x)} \geq \sum_{x \in S_p} p(x) \log \frac{1}{p(x)}, \text{ with equality if and only if } a(x) = p(x) \text{ for every } x \in S.$$

*When the weights come from  $p$  and the lengths  $\log(1/a)$  from any  $a$  satisfying the Kraft condition, the weighted average is smallest at  $a = p$ .*

## Proof

For  $t > 0$ ,  $\ln t \leq t - 1$ , with equality only at  $t = 1$ . Apply it at  $t = a(x)/p(x)$  for  $x \in S_p$ :

$$\sum_{x \in S_p} p(x) \ln \frac{a(x)}{p(x)} \leq \sum_{x \in S_p} p(x) \left( \frac{a(x)}{p(x)} - 1 \right) = \sum_{x \in S_p} a(x) - 1 \leq \sum_{x \in S} a(x) - 1 \leq 0.$$

Divide by  $\ln 2$ . Equality forces  $a = p$  on  $S_p$  (first inequality) and  $\sum_{S_p} a = 1$  (second and third), so  $a = 0 = p$  off  $S_p$ . □

# Source Coding Theorem: The Lower Bound

Proof that  $H(X) \leq L(c)$  for every prefix-free code  $c$

Put  $a(x) = 2^{-\ell_c(x)} > 0$  for  $x \in S$ . By Kraft,  $\sum_x a(x) \leq 1$ ; and  $\log \frac{1}{a(x)} = \ell_c(x)$ . Gibbs' lemma with  $p = p_X$  gives

$$L(c) = \sum_{x \in S_X} p_X(x) \ell_c(x) = \sum_{x \in S_X} p_X(x) \log \frac{1}{a(x)} \geq \sum_{x \in S_X} p_X(x) \log \frac{1}{p_X(x)} = H(X),$$

with equality if and only if  $2^{-\ell_c(x)} = p_X(x)$  for all  $x \in S$ : the probabilities are powers of 2 and the lengths are the informations. □

*The lengths of a prefix-free code define numbers  $2^{-\ell(x)}$  satisfying the Kraft condition, and the lemma says the average length under  $p_X$  is smallest when these numbers equal  $p_X$ .*

## The rover, in numbers

Any prefix-free code for four symbols has  $a = (2^{-\ell_1}, \dots, 2^{-\ell_4})$  with  $\sum a \leq 1$ , so  $L(c) = \frac{1}{2}\ell_1 + \frac{1}{4}\ell_2 + \frac{1}{8}\ell_3 + \frac{1}{8}\ell_4 = \sum_x p_X(x) \log \frac{1}{a(x)} \geq \sum_x p_X(x) \log \frac{1}{p_X(x)} = 1.75$ . The lengths  $(1, 2, 3, 3)$  give  $a = p_X$ , hence equality, and no other lengths do.

## Source Coding Theorem: The Upper Bound

### Proof that $L^* < H(X) + 1$ (Shannon's construction)

Take  $S_X = S$  (a symbol of probability 0 never occurs and needs no codeword). Round each ideal length up:  $\ell(x) = \lceil \log \frac{1}{p_X(x)} \rceil$ . Then  $2^{-\ell(x)} \leq p_X(x)$ , so  $\sum_x 2^{-\ell(x)} \leq \sum_x p_X(x) = 1$ , and Kraft supplies a prefix-free code  $c$  with these lengths. Its average length is

$$L(c) = \sum_x p_X(x) \left\lceil \log \frac{1}{p_X(x)} \right\rceil < \sum_x p_X(x) \left( \log \frac{1}{p_X(x)} + 1 \right) = H(X) + 1. \quad \square$$

*Rounding every ideal length up to a whole number wastes less than one bit per symbol.*

### Example: $p = (0.4, 0.3, 0.2, 0.1)$

Ideal lengths 1.32, 1.74, 2.32, 3.32; rounded up: 2, 2, 3, 4; Kraft sum  $\frac{1}{4} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} = \frac{11}{16}$ ; code 00, 01, 100, 1010 by the converse of Kraft.

$$L = 0.4 \cdot 2 + 0.3 \cdot 2 + 0.2 \cdot 3 + 0.1 \cdot 4 = 2.4 \text{ bits}, \quad H = 1.846, \quad L < H + 1 = 2.846.$$

Shannon's code is not optimal here: the lengths (1, 2, 3, 3) also satisfy Kraft and give  $L = 1.9$ . The theorem promises only a code within one bit of  $H$ .

# Entropy Adds Under Independence

## Theorem

If  $X \perp\!\!\!\perp Y$  then the pair  $(X, Y)$ , with pmf  $p_{X,Y}(x, y) = \mathbb{P}(X = x, Y = y)$ , has entropy  $H(X, Y) = H(X) + H(Y)$ .

## Proof

$p_{X,Y}(x, y) = p_X(x)p_Y(y)$ , so  $\log \frac{1}{p_{X,Y}(x,y)} = \log \frac{1}{p_X(x)} + \log \frac{1}{p_Y(y)}$ . Multiply by  $p_{X,Y}(x, y)$  and sum over  $(x, y)$ . In the first term,  $\sum_y p_{X,Y}(x, y) = p_X(x)$ , leaving  $\sum_x p_X(x) \log \frac{1}{p_X(x)} = H(X)$ ; likewise the second term gives  $H(Y)$ .  $\square$

*The information of a pair of independent values is the sum of the two informations, and expectation is linear.*

## Examples

Two dice,  $Y$  and  $Z$ :  $H(Y, Z) = \log 6 + \log 6 = \log 36 = 5.170$ , the uniform pmf on the 36 cells.  $k$  source symbols:  $H(X_1, \dots, X_k) = kH(X)$ , by induction.

- Unlike the product rule for expectations, this has a converse: Section 8 shows  $H(X, Y) \leq H(X) + H(Y)$  always, with equality only under independence.

## Closing the Gap: Code Blocks of Symbols

A source emitting A with probability 0.9 and B with probability 0.1 has  $h(0.9) = 0.469$  bits per symbol, yet every codeword has at least one bit; the gap is closed by coding several symbols at once.

### Theorem (block coding: entropy is the limit)

Let  $L_k^*$  be the best average length of a prefix-free code for blocks  $(X_1, \dots, X_k)$  of  $k$  source symbols. Then

$$H(X) \leq \frac{L_k^*}{k} < H(X) + \frac{1}{k}.$$

### Proof

The block is a symbol of the alphabet  $S^k$  with entropy  $kH(X)$  (previous slide). Apply the source coding theorem to it:  $kH(X) \leq L_k^* < kH(X) + 1$ . Divide by  $k$ . □

*Coding blocks of  $k$  symbols reduces the rounding loss per symbol to less than  $1/k$ , while the lower bound  $H(X)$  is unchanged.*

### The A/B source

Pairs have pmf (0.81, 0.09, 0.09, 0.01). The code 0, 10, 110, 111 costs  $0.81 + 0.18 + 0.27 + 0.03 = 1.29$  bits per pair, i.e. 0.645 bits per symbol, which is already below 1; the lower bound is 0.469.

## Cross-Entropy

The rover's mission changes: right is now the common instruction,  $p = (\frac{1}{8}, \frac{1}{8}, \frac{1}{4}, \frac{1}{2})$  for (up, down, left, right). The code 0, 10, 110, 111 stays installed.

$$\text{cost} = \frac{1}{8} \cdot 1 + \frac{1}{8} \cdot 2 + \frac{1}{4} \cdot 3 + \frac{1}{2} \cdot 3 = 2.625 \text{ bits per instruction,}$$

worse than the 2 bits of the fixed-length code, while the new entropy is still  $H(p) = 1.75$ .

### Definition

Let  $P, Q$  be pmfs on  $S$ , with  $S_P = \{x : P(x) > 0\}$ . The **cross-entropy** of  $P$  with  $Q$  is

$$H(P, Q) = \sum_{x \in S_P} P(x) \log \frac{1}{Q(x)} \quad \text{bits, with } \log \frac{1}{0} = \infty,$$

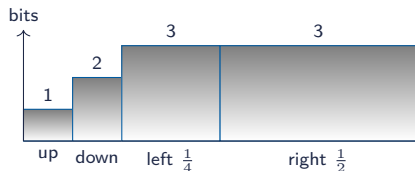
a function  $H: \Delta_n \times \Delta_n \rightarrow [0, \infty]$ . The first argument  $P$  is the *truth* and supplies the weights; the second,  $Q$ , is the *code* or *model* and supplies the lengths  $\log(1/Q(x))$ . Notation:  $H$  of two pmfs is cross-entropy;  $H(X, Y)$  of two *random variables* is the entropy of the pair (Section 4, Section 8).

*The average bits per symbol when the code was built for  $Q$  but the symbols come from  $P$ .*

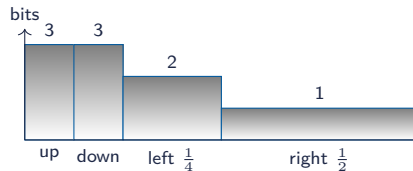
- Rover:  $Q = (\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \frac{1}{8})$  is what the code was built for,  $P = (\frac{1}{8}, \frac{1}{8}, \frac{1}{4}, \frac{1}{2})$  is the truth, and  $H(P, Q) = 2.625$ .

## Cross-Entropy: Widths From $P$ , Heights From $Q$

$H(P, Q) = 2.625$ : widths  $P$ , heights  $\log \frac{1}{Q}$



$H(P, P) = 1.75$ : widths  $P$ , heights  $\log \frac{1}{P}$



- $H(P, P) = H(P)$ : the code built for the truth. The lower bound of Section 4 reads  $H(P, Q) \geq H(P)$  for every  $Q$  (Gibbs' lemma with  $a = Q$ ).
- $H(P, Q) \neq H(Q, P)$  in general: swapping truth and code swaps widths and heights, which changes the area.
- $H(P, Q) = \infty$  if  $Q(x) = 0$  for some  $x$  with  $P(x) > 0$ : a code with no codeword for a symbol that does occur.

*The two pictures have the same widths; on the left the widest bar carries the tallest height, and the area grows from 1.75 to 2.625.*

## Cross-Entropy: Two Coins

### A uniform code on a lopsided truth

$Q = (\frac{1}{2}, \frac{1}{2})$ ,  $P = (0.9, 0.1)$ . Lengths  $\log 2 = 1$  and 1:  $H(P, Q) = 0.9 \cdot 1 + 0.1 \cdot 1 = 1$  bit. This holds for every  $P$ , since equal lengths cost 1 bit whatever the truth; as  $H(P) = h(0.1) = 0.469$ , the waste is 0.531 bits per symbol.

### A lopsided code on a fair truth

$Q = (0.9, 0.1)$ ,  $P = (\frac{1}{2}, \frac{1}{2})$ . Lengths  $\log \frac{1}{0.9} = 0.152$  and  $\log \frac{1}{0.1} = 3.322$ :

$$H(P, Q) = \frac{1}{2} \cdot 0.152 + \frac{1}{2} \cdot 3.322 = 1.737 \text{ bits,}$$

against  $H(Q) = 0.469$ , the average the code was designed for, and  $H(P) = 1$ , the entropy of the truth.

| truth $P$                    | code $Q$                     | $H(P, Q)$ | $H(P)$ | waste $H(P, Q) - H(P)$ |
|------------------------------|------------------------------|-----------|--------|------------------------|
| $(0.9, 0.1)$                 | $(\frac{1}{2}, \frac{1}{2})$ | 1         | 0.469  | 0.531                  |
| $(\frac{1}{2}, \frac{1}{2})$ | $(0.9, 0.1)$                 | 1.737     | 1      | 0.737                  |

*The uniform code wastes half a bit on the lopsided truth; the lopsided code wastes three quarters of a bit on the uniform truth, since its 3.3-bit codeword is used half the time.*

## Relative Entropy ( $D$ )

### Definition

For pmfs  $P, Q$  on  $S$ , the **relative entropy** (Kullback–Leibler divergence) of  $P$  from  $Q$  is

$$D(P \parallel Q) = H(P, Q) - H(P) = \sum_{x \in S_P} P(x) \log \frac{P(x)}{Q(x)} \quad \text{bits, with } \log \frac{P(x)}{0} = \infty,$$

a function  $D: \Delta_n \times \Delta_n \rightarrow [0, \infty]$ ;  $P$  is the truth,  $Q$  the code or model.

*The extra bits per symbol paid for coding with  $Q$  when the truth is  $P$ , over the minimum  $H(P)$ .*

### The rover

$D(P \parallel Q) = 2.625 - 1.75 = 0.875$  bits per instruction wasted by keeping the old code. Bar by bar,  $P(x) \left[ \log \frac{1}{Q(x)} - \log \frac{1}{P(x)} \right]$ :

$\frac{1}{8}(1-3) + \frac{1}{8}(2-3) + \frac{1}{4}(3-2) + \frac{1}{2}(3-1) = -0.25 - 0.125 + 0.25 + 1 = 0.875$ . Two bars became cheaper and two dearer; the dearer ones are the more probable, so the total is positive.

- Two coins:  $D((0.9, 0.1) \parallel (\frac{1}{2}, \frac{1}{2})) = 0.531$  and  $D((\frac{1}{2}, \frac{1}{2}) \parallel (0.9, 0.1)) = 0.737$ , the waste column of the previous table.

# Gibbs' Inequality

## Theorem (Gibbs' inequality)

$D(P \parallel Q) \geq 0$ , with equality if and only if  $P = Q$ . Equivalently,  $H(P, Q) \geq H(P)$ , with equality iff  $Q = P$ .

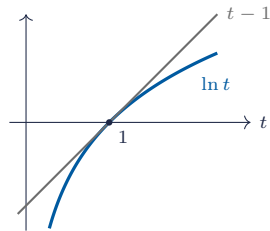
*No code has smaller average length than the code built for the true distribution, and no model achieves a smaller average cost on a source than the source's own distribution.*

## Proof

If  $Q(x) = 0 < P(x)$  for some  $x$  then  $D = \infty > 0$ . Otherwise, with  $\ln t \leq t - 1$  at  $t = Q(x)/P(x)$  (the lemma of Section 4 with  $a = Q$ ):

$$\begin{aligned} -D(P \parallel Q) \ln 2 &= \sum_{x \in S_P} P(x) \ln \frac{Q(x)}{P(x)} \leq \sum_{x \in S_P} P(x) \left( \frac{Q(x)}{P(x)} - 1 \right) \\ &= \sum_{x \in S_P} Q(x) - 1 \leq 0, \end{aligned}$$

with equality iff  $Q = P$  on  $S_P$  and  $\sum_{S_P} Q = 1$ , i.e.  $Q = P$ .  $\square$



$\ln t \leq t - 1$ , with equality only at  $t = 1$ , is the only inequality the proof uses.

# Consequences of Gibbs' Inequality

## Corollary 1: the maximum of entropy

If  $X$  takes  $n$  values then  $H(X) \leq \log n$ , with equality iff  $X$  is uniform.

## Proof

Take  $Q$  uniform,  $Q(x) = \frac{1}{n}$ :  $0 \leq D(p_X \| Q) = \sum_x p_X(x) \log(n p_X(x)) = \log n - H(X)$ , with equality iff  $p_X = Q$ .  $\square$

*The uniform distribution has the largest entropy on  $n$  values, and the fixed-length code is optimal for it and for no other source.*

## Corollary 2: cross-entropy is minimised by the truth

For fixed  $P$ , the function  $Q \mapsto H(P, Q)$  on  $\Delta_n$  has its unique minimum at  $Q = P$ , where it equals  $H(P)$ . This is the property that allows cross-entropy to serve as a training loss (Section 6).

## Corollary 3, for Section 8

Mutual information is non-negative:  $I(X; Y) = D(p_{X,Y} \| p_X p_Y) \geq 0$ , with equality iff  $X \perp\!\!\!\perp Y$ .

- Section 8 proves Corollary 3.

## What Relative Entropy Is Not: A Distance

A distance  $d$  has  $d(P, Q) = d(Q, P)$ ,  $d(P, R) \leq d(P, Q) + d(Q, R)$ ,  $d < \infty$ , and  $d(P, Q) = 0$  iff  $P = Q$ . Relative entropy has only the last.

| property            | fails at                                                           | numbers                                                                                                                                                                     |
|---------------------|--------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| symmetry            | $P = (\frac{1}{2}, \frac{1}{2}), Q = (\frac{1}{4}, \frac{3}{4})$   | $D(P\ Q) = 1.2075 - 1 = 0.2075$ : the 2-bit codeword comes half the time instead of a quarter.<br>$D(Q\ P) = 1 - 0.8113 = 0.1887$ : 1 bit always, against the 0.811 needed. |
| triangle inequality | $P = (\frac{1}{2}, \frac{1}{2}), Q = (0.9, 0.1), R = (0.99, 0.01)$ | $D(P\ Q) + D(Q\ R) = 0.737 + 0.208 = 0.945 < 2.329 = D(P\ R)$ .                                                                                                             |
| finiteness          | $Q(x) = 0 < P(x)$                                                  | $D(P\ Q) = \infty$ : the model assigned probability 0 to an outcome that occurred; practical models keep every probability strictly positive.                               |
| $D = 0$ iff $P = Q$ | holds (Gibbs)                                                      | $D$ is zero only for identical pmfs; its value is the waste for one direction of use.                                                                                       |

*Relative entropy is a divergence rather than a distance; it measures the cost of using  $Q$  when the truth is  $P$ , and this cost is not symmetric in the two.*

## Why the Loss Is a Logarithm

A classifier outputs a pmf  $q \in \Delta_n$  over  $n \geq 3$  classes; the true class is drawn from  $p \in \Delta_n$ . When class  $y$  occurs, charge  $f(q_y)$ , with  $f: (0, 1] \rightarrow \mathbb{R}$  decreasing and differentiable on  $(0, 1)$ .

### Theorem

Suppose  $f$  is **honest**: for every truth  $p$ , the expected penalty  $\sum_y p_y f(q_y)$  over  $q \in \Delta_n$  is smallest at  $q = p$  and nowhere else. Then  $f(t) = -c \log t + d$  on  $(0, 1)$  with  $c > 0$ ; normalised,  $f(t) = -\log t$ , the information of the outcome under the model.

### Proof

Take  $p$  with every  $p_y > 0$ . Then  $q = p$  is a minimiser in the interior of  $\Delta_n$ , so Lagrange applies there:  $p_y f'(p_y) = \lambda$  for all  $y$ , i.e.  $t f'(t)$  takes the same value at every coordinate  $t = p_y$ . For  $s, t > 0$  with  $s + t < 1$  use  $p = (s, t, \frac{1-s-t}{n-2}, \dots, \frac{1-s-t}{n-2})$  (possible as  $n \geq 3$ ):  $s f'(s) = t f'(t)$ . For  $s + t \geq 1$  compare both with a small  $u$ . So  $t f'(t) = -c$  is constant on  $(0, 1)$ ,  $f(t) = -c \ln t + d$ , and decreasing means  $c > 0$ .  $\square$

*If the penalty is to be minimised by reporting the true distribution, it must be the logarithm of the probability assigned to the outcome that occurred, up to scale and shift.*

## What Goes Wrong With Any Other Penalty

Linear penalty  $f(t) = 1 - t$

Expected penalty  $1 - \sum_y p_y q_y$ , smallest when all of  $q$  sits on the most probable class: for  $p = (0.5, 0.3, 0.2)$  the minimiser is  $q = (1, 0, 0)$ , penalty 0.5, while reporting  $q = p$  costs 0.62. The penalty is smallest when the probable class is overstated.

Squared penalty on the realised probability,  $f(t) = (1 - t)^2$

Lagrange:  $-2p_y(1 - q_y) = \lambda$ , so  $q_y = 1 + \lambda/(2p_y)$ . For  $p = (0.5, 0.3, 0.2)$ :  $q = (0.613, 0.355, 0.032) \neq p$ . Again the probable class is overstated and the improbable one understated. (The Brier score penalises the entire vector  $q$  rather than  $q_y$  alone, and is honest.)

Logarithmic penalty  $f(t) = -\log t$

Expected penalty  $\sum_y p_y \log \frac{1}{q_y} = H(p, q) = H(p) + D(p||q)$ , minimised at  $q = p$  by Gibbs, and only there. For  $p = (0.5, 0.3, 0.2)$  the minimum is  $H(p) = 1.485$  bits.

*Among penalties that depend only on  $q_y$ , the logarithm is the only one minimised by reporting the truth.*

# Cross-Entropy Loss

## Definition (cross-entropy loss)

Data  $(x_1, y_1), \dots, (x_N, y_N)$ , labels among  $n$  classes; a model with parameters  $\theta \in \Theta \subseteq \mathbb{R}^d$  outputs a pmf  $q_\theta(\cdot | x) \in \Delta_n$  over the classes for each input  $x$ . Its loss is

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \log \frac{1}{q_\theta(y_i | x_i)} \quad \text{bits per example.}$$

*Each example is charged the information of its true label under the model, which is the codeword length the model would assign to it.*

## What minimising it does

For an input  $x$  whose true label has pmf  $p(\cdot | x)$ , the loss averages to  $H(p(\cdot | x), q_\theta(\cdot | x)) = H(p(\cdot | x)) + D(p(\cdot | x) \parallel q_\theta(\cdot | x))$ . The first term does not depend on  $\theta$ . Minimising the loss therefore minimises  $D$ , and by Gibbs' inequality the minimum is attained only when the model equals the truth.

- The loss cannot fall below  $H(p)$ : a training curve that levels off at 0.5 bits has reached its minimum if the labels themselves carry 0.5 bits of uncertainty per example.

## Distillation: Training on a Full Distribution

A large *teacher* model gives, for each input  $x$ , a full pmf  $p(\cdot | x)$  over classes. A small *student*  $q_\theta$  is trained to minimise

$$H(p(\cdot | x), q_\theta(\cdot | x)) = \sum_y p(y | x) \log \frac{1}{q_\theta(y | x)} \quad \text{averaged over inputs.}$$

*The same cross-entropy loss, with the teacher's soft probabilities in place of a single correct label.*

- A one-hot label has entropy 0 and records only which class occurred; the teacher's pmf has entropy  $H(p) > 0$  and also records how the alternatives rank, which the student can learn from.
- The minimum of the loss is  $H(p)$  rather than 0; the part the student can reduce is  $D(p||q_\theta)$ .

### Numbers

Teacher  $p = (0.7, 0.2, 0.1)$ , student  $q = (0.8, 0.1, 0.1)$ :

$H(p, q) = 0.7 \log \frac{1}{0.8} + 0.2 \log \frac{1}{0.1} + 0.1 \log \frac{1}{0.1} = 1.222 \text{ bits} = H(p) + D(p||q) = 1.157 + 0.065$ . The student is 0.065 bits from the teacher; the remaining 1.157 is the teacher's own uncertainty.

# Prediction and Compression Are Equivalent

## Theorem (arithmetic coding; statement only)

Let a model assign to a text  $x_1 \dots x_n$  the conditional probabilities  $q(x_i \mid x_1 \dots x_{i-1}) > 0$ . There is a uniquely decodable encoding of the text in fewer than

$$\sum_{i=1}^n \log \frac{1}{q(x_i \mid x_1 \dots x_{i-1})} + 2 \quad \text{bits},$$

decodable by anyone who has the same model.

*Any predictor yields a compressor in which each symbol costs the information the model assigned to it, so the total length is the model's cross-entropy loss on the text plus at most two bits.*

- The training loss of a language model, in bits per token, is the length per token to which it compresses its training text; a lower loss corresponds to a smaller compressed file.
- Conversely a compressor defines a predictor: a symbol coded with  $\ell$  bits has been assigned probability  $2^{-\ell}$ . By Gibbs' inequality, no model gets below the source's entropy rate (Section 9) on average.
- The rover: the model “up  $\frac{1}{2}$ , down  $\frac{1}{4}$ , left, right  $\frac{1}{8}$ ” compresses instructions to 1.75 bits each; the reversed model costs 2.625 on the same stream.

## Motivation: Which Wordle Opener?

Wordle: a five-letter answer is drawn from a list of 2315 words. Each guess returns five colours, ■ right letter in the right place, ■ right letter in the wrong place, ■ letter absent: one of  $3^5 = 243$  patterns.

### Uncertainty before the first guess

Before the first guess the answer is uniform on 2315 words, an uncertainty of  $\log 2315 = 11.18$  bits. A natural measure of a guess is the amount by which it reduces this, but the reduction depends on which of the 243 patterns comes back.

### Questions we cannot yet answer

1. Is weary a good first guess? Is slate better? Is anything better than that?
2. Can any strategy average fewer than 3 guesses?
3. Why did the “best opener” change when people optimised for the actual score instead of information?

*The relevant quantity is the entropy of the pattern distribution of a guess, which is the expected information the guess delivers and which Section 8 identifies as a mutual information.*

- Source of the real-game numbers below: 3Blue1Brown, “Solving Wordle using information theory” (2022).

## A Guess Is a Partition of the Remaining Answers

### Definition

Let  $W$  be the set of remaining possible answers,  $|W| = N$ , and let the answer  $A: \Omega \rightarrow W$  be uniform on  $W$ :  $\mathbb{P}(A = w) = \frac{1}{N}$ . Fix a guess  $g$ . The **pattern function** of  $g$  is  $\pi_g: W \rightarrow \{\text{grey}, \text{orange}, \text{green}\}^5$ , sending each possible answer  $w$  to the colours  $g$  would receive if  $w$  were the answer. The **pattern** is the random variable  $\Pi = \pi_g(A): \Omega \rightarrow \{\text{grey}, \text{orange}, \text{green}\}^5$ , with pmf

$$p_{\Pi}(\pi) = \mathbb{P}(\Pi = \pi) = \frac{|\{w \in W : \pi_g(w) = \pi\}|}{N} = \frac{|\text{bucket of } \pi|}{N}.$$

*A guess sorts the remaining answers into buckets, one per pattern, and the pattern returned identifies the bucket that contains the answer.*

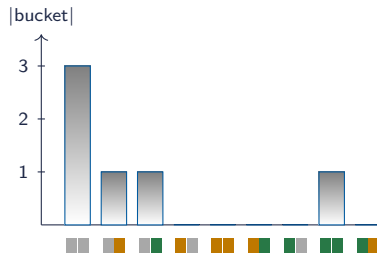
- Seeing pattern  $\pi$  shrinks the candidate list from  $N$  to  $|\text{bucket of } \pi| = N p_{\Pi}(\pi)$ . The information of the pattern is  $I_{\Pi}(\pi) = \log \frac{1}{p_{\Pi}(\pi)} = \log \frac{N}{|\text{bucket}|}$ : the number of halvings of the list.
- The guess is fully described by its pattern distribution  $p_{\Pi}$ : a bar chart with 243 bars. Two guesses with the same bar chart are equally informative, whatever the letters.
- A pattern with a bucket of size 1 carries  $\log N$  bits and ends the game next turn; the all-grey pattern usually has a large bucket and carries little information.

## Toy Wordle: Two-Letter Words, Done by Hand

$W = \{\text{AM, IN, NO, OF, ON, TO}\}$ ,  $N = 6$ , two-letter words,  
 $3^2 = 9$  possible patterns. Guess  $g = \text{IN}$ ; for each possible  
 answer, the pattern:

| answer $w$ | $\pi_{\text{IN}}(w)$ | why            |
|------------|----------------------|----------------|
| AM         | ■                    | no I, no N     |
| IN         | ■                    | the answer     |
| NO         | ■ ■                  | N, wrong place |
| OF         | ■                    |                |
| ON         | ■ ■                  | N in place     |
| TO         | ■                    |                |

Buckets: ■ {AM, OF, TO}, and ■, ■ ■, ■ ■ with one word  
 each. So  $p_{\Pi} = (\frac{3}{6}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6})$  on these four patterns, 0 on the  
 other five.



The bar chart of the guess IN: all nine  
 patterns, four of them possible. Bucket of  
 3:  $I = \log \frac{6}{3} = 1$  bit. Buckets of 1:  
 $I = \log 6 = 2.585$  bits.

# The Entropy of a Guess Is Its Expected Information

## Definition

The **entropy of the guess**  $g$  is the entropy of its pattern,  $H(\Pi) = \sum_{\pi} p_{\Pi}(\pi) \log \frac{1}{p_{\Pi}(\pi)} = \mathbb{E}[I_{\Pi}(\Pi)]$ , the expected number of halvings of the list.

*The expected information over the patterns that might be returned, evaluated before the guess is made.*

## Toy Wordle

| guess    | bucket sizes     | $H(\Pi)$                                                                                           |
|----------|------------------|----------------------------------------------------------------------------------------------------|
| NO or ON | 1, 1, 1, 1, 1, 1 | $\log 6 = 2.585$ bits, the maximum: each pattern names the answer                                  |
| OF or TO | 2, 2, 1, 1       | $2 \cdot \frac{2}{6} \cdot 1.585 + 2 \cdot \frac{1}{6} \cdot 2.585 = 1.918$ bits                   |
| IN       | 3, 1, 1, 1       | $\frac{3}{6} \cdot 1 + 3 \cdot \frac{1}{6} \cdot 2.585 = 1.792$ bits                               |
| AM       | 5, 1             | $\frac{5}{6} \cdot 0.263 + \frac{1}{6} \cdot 2.585 = 0.650$ bits; little learned five times in six |

## Real Wordle, $N = 2315$ (3Blue1Brown)

weary 4.90 bits; slate 5.87; soare 5.89, the best by this measure, leaving  $2315/2^{5.89} \approx 39$  candidates on average. The ceiling  $\log 243 = 7.92$  (one word per pattern) is unreachable since  $2315 > 243$ .

## Back to the Openers: Two Guesses Almost Never Finish

Let  $G: \Omega \rightarrow \{1, 2, \dots\}$  be the number of guesses a strategy takes to hit the answer; the score of the strategy is  $\mathbb{E}[G]$ .

### Claim

For every strategy,  $\mathbb{P}(G \leq 2) \leq \frac{243}{2315} = 0.105$ , and hence  $\mathbb{E}[G] \geq 3 - 2\mathbb{P}(G = 1) - \mathbb{P}(G = 2) \geq 3 - \frac{244}{2315} = 2.89$ .

### Proof

$G = 1$  for at most one answer, the first guess itself.  $G = 2$  requires the second guess to be the answer; the second guess is a function of the first pattern, which takes at most 242 non-winning values, so at most 242 answers have  $G = 2$ . Then  $\mathbb{E}[G] \geq 1 \cdot \mathbb{P}(G = 1) + 2\mathbb{P}(G = 2) + 3\mathbb{P}(G \geq 3)$ .  $\square$

*A guess can only single out one answer per pattern it can receive, so two guesses cover at most 243 of 2315 answers.*

- In bits:  $\log 2315 = 11.18$ ; the best opening pair extracts 10.04 bits on average (3Blue1Brown), leaving 1.14 bits, i.e.  $2^{1.14} \approx 2.2$  candidates for guess three. The best strategy known averages 3.42 guesses.
- Toy Wordle: open NO,  $\mathbb{E}[G] = \frac{1}{6} \cdot 1 + \frac{5}{6} \cdot 2 = 1.83$ ; open IN then OF on ,  $\mathbb{E}[G] = \frac{1+2+2+2+3+3}{6} = 2.17$ . Here the entropy ranking and the score ranking agree.

## Back to the Openers: Entropy Is a Proxy for the Score

### Bits versus guesses

$H(\Pi)$  measures what the guess reveals;  $\mathbb{E}[G]$  counts turns. They differ because

- a guess  $g \in W$  can end the game now ( $\mathbb{P}(G = \text{this turn}) = \frac{1}{N}$ ); a guess  $g \notin W$  cannot, however large  $H(\Pi)$ ;
- on the last turn only whether the list has size 1 matters, not how many bits are gained.

Maximising  $H(\Pi)$  each turn is a proxy for minimising  $\mathbb{E}[G]$ .

*Entropy measures the information gained per guess while the game counts guesses, and the two rankings agree closely but not perfectly.*

- Section 8 identifies  $H(\Pi)$  as the mutual information  $I(A; \Pi)$  between the answer and the pattern, which gives a precise meaning to the phrase “expected information of a guess”.

| opener | $H(\Pi)$   | $\mathbb{E}[G]$ , best play |
|--------|------------|-----------------------------|
| soare  | 5.89 (max) |                             |
| slate  | 5.87       |                             |
| salet  | < 5.89     | 3.42 (min)                  |
| crane  | < 5.89     | 3.43                        |
| weary  | 4.90       |                             |

Scores by full game-tree search (3Blue1Brown, 2022): the best opener by score is not the best by entropy.

# The Two-Dice Table, and Joint Entropy

Two fair dice,  $\mathbb{P}(\{\omega\}) = \frac{1}{36}$  on  $\Omega = \{1, \dots, 6\}^2$ .  $Y$  the first die (row),  $Z$  the second (column),  $X = Y + Z$  (the cell).  
The rows are the events  $\{Y = y\}$ .

## Definition

The **joint entropy** of  $X$  and  $Y$  is the entropy of the pair, i.e. of  $p_{X,Y}: S_X \times S_Y \rightarrow [0, 1]$ ,  
 $p_{X,Y}(x, y) = \mathbb{P}(X = x, Y = y)$ :

$$H(X, Y) = \sum_{x,y} p_{X,Y}(x, y) \log \frac{1}{p_{X,Y}(x, y)}.$$

*The bits needed to report both values at once.*

## Two dice

$p_{X,Y}(x, y) = \frac{1}{36}$  if  $x - y \in \{1, \dots, 6\}$ , else 0: one cell per pair that occurs. So  $(X, Y)$  is uniform on 36 pairs and  $H(X, Y) = \log 36 = 5.170$  bits; the sum and the first die together determine the cell.

| $y \setminus z$ | 1 | 2 | 3 | 4  | 5  | 6  |
|-----------------|---|---|---|----|----|----|
| 1               | 2 | 3 | 4 | 5  | 6  | 7  |
| 2               | 3 | 4 | 5 | 6  | 7  | 8  |
| 3               | 4 | 5 | 6 | 7  | 8  | 9  |
| 4               | 5 | 6 | 7 | 8  | 9  | 10 |
| 5               | 6 | 7 | 8 | 9  | 10 | 11 |
| 6               | 7 | 8 | 9 | 10 | 11 | 12 |

Rows  $y$ , columns  $z$ , cell  $x = y + z$ . Row 4 is the event  $\{Y = 4\}$ .

## $H(X \mid Y = y)$ : The Entropy of a Row

### Definition

For  $y \in S_Y$ , let  $p_{X|Y}(\cdot \mid y)$  be the conditional pmf of  $X$  given  $Y = y$ , the row's distribution:  $p_{X|Y}(x \mid y) = p_{X,Y}(x, y)/p_Y(y)$ . The **conditional entropy of  $X$  given  $Y = y$**  is the function

$$S_Y \rightarrow [0, \infty), \quad y \mapsto H(X \mid Y = y) = \sum_x p_{X|Y}(x \mid y) \log \frac{1}{p_{X|Y}(x \mid y)}.$$

*The entropy of the conditional distribution in row  $y$ , giving one number for each row.*

### Two dice, $X$ the sum

Row  $y$  contains the six distinct sums  $y + 1, \dots, y + 6$ , each with conditional probability  $\frac{1}{6}$ . So

$H(X \mid Y = y) = \log 6 = 2.585$  bits for every  $y$ .

- The other way round:  $\{X = x\}$  is an anti-diagonal of  $n_x$  cells with  $Y$  uniform on it, so  $H(Y \mid X = x) = \log n_x$ , which runs 0, 1, 1.585, 2, 2.322, 2.585 and back down as  $x$  goes from 2 to 12.

| $y \setminus z$ | 1 | 2 | 3 | 4  | 5  | 6  |
|-----------------|---|---|---|----|----|----|
| 1               | 2 | 3 | 4 | 5  | 6  | 7  |
| 2               | 3 | 4 | 5 | 6  | 7  | 8  |
| 3               | 4 | 5 | 6 | 7  | 8  | 9  |
| 4               | 5 | 6 | 7 | 8  | 9  | 10 |
| 5               | 6 | 7 | 8 | 9  | 10 | 11 |
| 6               | 7 | 8 | 9 | 10 | 11 | 12 |

$\{X = 7\}$ : six cells,  $H(Y \mid X = 7) = \log 6$ .

## $H(X | Y)$ : The Average Over the Rows

### Definition

The **conditional entropy of  $X$  given  $Y$**  is the row entropy averaged over the rows with their probabilities,

$$H(X | Y) = \sum_{y \in S_Y} p_Y(y) H(X | Y = y) = \sum_{x,y} p_{X,Y}(x,y) \log \frac{1}{p_{X|Y}(x | y)} \quad \text{a number.}$$

*The uncertainty remaining in  $X$  once  $Y$  is known, averaged over the value of  $Y$ ; the row entropies are combined as in the law of total expectation.*

- **What it is not.**  $\mathbb{E}[X | Y]$  is a random variable, but  $H(X | Y)$  is a *number*, the row entropies having been averaged; the notation is parallel but the objects are of different kinds.

### Two dice

$H(X | Y) = \sum_y \frac{1}{6} \log 6 = 2.585$  bits: every row of sums is uniform on six values.

$H(Y | X) = \sum_x \frac{n_x}{36} \log n_x = 2 \left[ \frac{2}{36} \cdot 1 + \frac{3}{36} \cdot 1.585 + \frac{4}{36} \cdot 2 + \frac{5}{36} \cdot 2.322 \right] + \frac{6}{36} \cdot 2.585 = 1.896$  bits:  
given the sum, the uncertainty about the first die is less than the  $\log 6$  bits of an unobserved die.

# The Chain Rule

## Theorem (chain rule)

$$H(X, Y) = H(Y) + H(X | Y) = H(X) + H(Y | X).$$

## Proof

$p_{X,Y}(x, y) = p_Y(y) p_{X|Y}(x | y)$ , so  $\log \frac{1}{p_{X,Y}(x,y)} = \log \frac{1}{p_Y(y)} + \log \frac{1}{p_{X|Y}(x|y)}$ . Multiply by  $p_{X,Y}(x, y)$  and sum over  $(x, y)$ . The first term:  $\sum_y \log \frac{1}{p_Y(y)} \sum_x p_{X,Y}(x, y) = \sum_y p_Y(y) \log \frac{1}{p_Y(y)} = H(Y)$ . The second term is  $H(X | Y)$  by definition. Swap the roles of  $X$  and  $Y$  for the other form.  $\square$

*Reporting the cell as the row followed by the position within the row costs  $H(Y)$  plus  $H(X | Y)$  bits.*

## Two dice, both ways

$$\log 36 = 5.170 = \underbrace{2.585}_{H(Y)} + \underbrace{2.585}_{H(X|Y)} = \underbrace{3.274}_{H(X)} + \underbrace{1.896}_{H(Y|X)}.$$

- Extremes:  $X \perp\!\!\!\perp Y$  gives  $H(X | Y) = H(X)$  and  $H(X, Y) = H(X) + H(Y)$  (Section 4);  $X$  a function of  $Y$  gives  $H(X | Y) = 0$  and  $H(X, Y) = H(Y)$ .

# Mutual Information ( $I$ )

## Definition

The **mutual information** of  $X$  and  $Y$  is the uncertainty in  $X$  removed by learning  $Y$ :

$$I(X; Y) = H(X) - H(X | Y) \quad \text{bits, a number.}$$

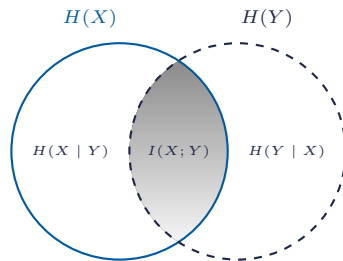
*The number of bits of  $X$  that  $Y$  carries, computed as the uncertainty about  $X$  before  $Y$  is seen minus the uncertainty remaining afterwards.*

## Two dice

$I(X; Y) = H(X) - H(X | Y) = 3.274 - 2.585 = 0.689$  bits: the first die carries about two thirds of a bit about the sum. The other way:

$I(Y; X) = H(Y) - H(Y | X) = 2.585 - 1.896 = 0.689$ . The two agree; the next slide shows that this always holds.

- Two dice in the picture:  $H(X, Y) = 5.170$ , split as  $2.585 + 0.689 + 1.896$ .



The union of the two circles is  $H(X, Y)$ . Left circle  $H(X)$ , right circle  $H(Y)$ , overlap  $I(X; Y)$ .

## Mutual Information: Properties

### Theorem

1.  $I(X; Y) = H(X) + H(Y) - H(X, Y) = I(Y; X)$ .
2.  $I(X; Y) = D(p_{X,Y} \parallel p_X p_Y) = \sum_{x,y} p_{X,Y}(x, y) \log \frac{p_{X,Y}(x, y)}{p_X(x) p_Y(y)} \geq 0$ , with equality iff  $X \perp\!\!\!\perp Y$ .
3.  $H(X | Y) \leq H(X)$ , with equality iff  $X \perp\!\!\!\perp Y$ ; and  $H(X, Y) \leq H(X) + H(Y)$ .
4.  $I(X; X) = H(X)$ , and  $I(X; Y) \leq \min(H(X), H(Y))$ .

### Proof

(1) Chain rule:  $H(X | Y) = H(X, Y) - H(Y)$ ; substitute; the result is symmetric. (2)  $H(X) - H(X | Y) = \sum_{x,y} p_{X,Y}(x, y) \left[ \log \frac{1}{p_X(x)} - \log \frac{p_Y(y)}{p_{X,Y}(x, y)} \right]$ , the displayed sum, which is  $D$  of the joint pmf from the product pmf; Gibbs gives  $\geq 0$ , with equality iff  $p_{X,Y} = p_X p_Y$ : independence. (3) Rearrange (2) and (1). (4)  $H(X | X) = 0$ ;  $H(X | Y) \geq 0$  gives  $I \leq H(X)$ , and symmetrically  $I \leq H(Y)$ .  $\square$

*Conditioning on  $Y$  never increases the uncertainty about  $X$  on average, although a single row can ( $M = \max(Y, Z)$ :  $H(M) = 2.320$  but  $H(M | Y = 1) = \log 6 = 2.585$ ), and it leaves the uncertainty unchanged only when every row has the distribution of the full table.*

## Example: The Binary Symmetric Channel

A bit  $X$ , uniform on  $\{0, 1\}$ , is sent down a wire that flips it with probability  $\varepsilon$ , independently of  $X$ ;  $Y$  is what arrives. We compute how many bits about  $X$  one received bit carries.

- One might expect  $1 - \varepsilon$  bits; the computation below gives less.
- Rows  $\{X = 0\}$  and  $\{X = 1\}$ : each row's distribution for  $Y$  is  $(1 - \varepsilon, \varepsilon)$  or  $(\varepsilon, 1 - \varepsilon)$ , entropy  $h(\varepsilon)$ . So  $H(Y | X = x) = h(\varepsilon)$  for both rows and  $H(Y | X) = h(\varepsilon)$ .
- $Y$  is uniform by symmetry:  $H(Y) = 1$ .

$I(X; Y) = H(Y) - H(Y | X) = 1 - h(\varepsilon)$  bits per received bit.

| $p_{X,Y}$     | $Y = 0$                   | $Y = 1$                   |
|---------------|---------------------------|---------------------------|
| $X = 0$       | $\frac{1-\varepsilon}{2}$ | $\frac{\varepsilon}{2}$   |
| $X = 1$       | $\frac{\varepsilon}{2}$   | $\frac{1-\varepsilon}{2}$ |
| <hr/>         |                           |                           |
| $\varepsilon$ | $h(\varepsilon)$          | $I(X; Y)$                 |
| <hr/>         |                           |                           |
| 0             | 0                         | 1                         |
| 0.01          | 0.081                     | 0.919                     |
| 0.1           | 0.469                     | 0.531                     |
| 0.25          | 0.811                     | 0.189                     |
| 0.5           | 1                         | 0                         |

*A wire that flips one bit in ten delivers about half a bit of information per bit sent rather than nine tenths, because the receiver does not know which bits were flipped.*

- $1 - h(\varepsilon)$  is the capacity of this channel (Shannon's channel coding theorem, not proved here): the rate at which reliable communication is possible over it.

## Back to Wordle: The Expected Information of a Guess Is $I(A; \Pi)$

Answer  $A$  uniform on  $W$ ,  $|W| = N$ ; guess  $g$ ; pattern  $\Pi = \pi_g(A)$ . Rows  $\{A = w\}$ , one per answer; the pattern is constant on each.

### Theorem

$I(A; \Pi) = H(\Pi)$ : the entropy of the guess (Section 7) is the mutual information between answer and pattern.

### Proof

$\Pi$  is a function of  $A$ : row  $\{A = w\}$  has the single pattern  $\pi_g(w)$ , so  $H(\Pi \mid A = w) = 0$  for every  $w$ ,  $H(\Pi \mid A) = 0$ , and  $I(A; \Pi) = H(\Pi) - 0$ . By symmetry also

$I(A; \Pi) = H(A) - H(A \mid \Pi) = \log N - \sum_{\pi} p_{\Pi}(\pi) \log |\text{bucket of } \pi|$ : the expected shrinkage of the list, in bits. □

*The pattern is determined by the answer, so all of its entropy is information about the answer.*

### Toy Wordle, guess IN

$H(A) = \log 6 = 2.585$ . With probability  $\frac{1}{2}$  the bucket has size 3 (entropy  $\log 3 = 1.585$ ), else size 1 (entropy 0):  $H(A \mid \Pi) = 0.792$  and  $I(A; \Pi) = 2.585 - 0.792 = 1.792 = H(\Pi)$ , as in Section 7.

## Motivation: Why Thirty Letters Break a Substitution Cipher

A **substitution cipher** replaces each letter by another under a fixed bijection  $k$  of the alphabet:  $26! \approx 4 \cdot 10^{26}$  keys,  $\log 26! = 88.4$  bits, so exhaustive search is infeasible. Yet a ciphertext of twenty-five to thirty letters usually has a unique sensible decryption, and a longer one can be broken by frequency counts in minutes.

### Questions we cannot yet answer

1. How many bits does one letter of English carry? The count  $\log 27 = 4.75$  (26 letters and a space) is too large by a factor of four or five.
2. How does a model of text turn that number into a decryption, a compression ratio, or a language-model score?
3. Why thirty letters, and not three or three hundred?

*The tools are a Markov model of letters, its entropy rate, and the cross-entropy of a model on real text.*

- Shannon, “Prediction and Entropy of Printed English” (1951), is the source of every English figure quoted below.

## Three Models of Text

Text is a sequence of random variables  $X_1, X_2, \dots : \Omega \rightarrow S$ ,  $|S| = 27$  (letters and space). A model specifies the joint pmf of every stretch  $X_1 \dots X_n$ .

### Definition

1. **Monographic (order 1).** A pmf  $p: S \rightarrow [0, 1]$  of single letters; the  $X_i$  are i.i.d. with pmf  $p$ :  
 $\mathbb{P}(X_1 = x_1, \dots, X_n = x_n) = \prod_i p(x_i)$ .
2. **Digraphic (order 2).** A pmf  $p_2: S \times S \rightarrow [0, 1]$  of adjacent pairs,  
 $p_2(x, x') = \mathbb{P}(X_i = x, X_{i+1} = x')$ , the same for every  $i$ ; its marginal is  $p$ .
3. **Markov. A transition matrix**  $Q: S \times S \rightarrow [0, 1]$  with  $\sum_{x'} Q(x, x') = 1$  for every  $x$ , where  
 $Q(x, x') = \mathbb{P}(X_{i+1} = x' \mid X_i = x) = p_2(x, x')/p(x)$  for  $p(x) > 0$  (any row for  $p(x) = 0$ ), and

$$\mathbb{P}(X_1 = x_1, \dots, X_n = x_n) = p(x_1) Q(x_1, x_2) Q(x_2, x_3) \cdots Q(x_{n-1}, x_n).$$

*The first model records that E is common and Z rare, the second that TH and HE are common and Q is followed by U, and the third converts the pair table into a rule for generating text one letter at a time.*

- $Q$  is a table like the two-dice table: row  $x$  corresponds to the event  $\{X_i = x\}$ , the entries along it form the conditional pmf of the next letter, and conditioning on the previous letter amounts to restricting to a row.

# Entropy Rate of a Markov Model

## Definition

A pmf  $\pi$  on  $S$  is **stationary** for  $Q$  if  $\sum_x \pi(x)Q(x, x') = \pi(x')$  for all  $x'$ : letter frequencies do not drift. For the Markov model started from  $\pi$ , the **entropy rate** is

$$H(X_{i+1} \mid X_i) = \sum_{x \in S} \pi(x) H(X_{i+1} \mid X_i = x) = \sum_{x \in S} \pi(x) H(Q(x, \cdot)) \quad \text{bits per letter,}$$

the entropy of row  $x$  of  $Q$ , averaged over rows with the row probabilities  $\pi(x)$ .

*The bits needed for the next letter when the previous one is known, obtained as for  $H(X \mid Y)$  by averaging the row entropies of the transition table.*

- **What it is not.** It differs from  $H(\pi)$ , the entropy of a letter seen alone, and from the entropy of any single row; it is the average of the row entropies, and equals  $H(X_{i+1} \mid X_i)$  for every  $i$  because  $\pi$  is stationary.
- $n$  letters have entropy  $H(X_1, \dots, X_n) = H(\pi) + (n-1)H(X_{i+1} \mid X_i)$ , by the chain rule for  $n$  variables (by induction) and the Markov property  $H(X_{k+1} \mid X_1, \dots, X_k) = H(X_{k+1} \mid X_k)$ : each letter depends on the past only through the last letter (stated without proof). Per letter this tends to the entropy rate, the lower bound for compressing the model's text (Section 4 applied to blocks).

# Entropy Rate at Most Monographic Entropy

## Theorem

$H(X_{i+1} | X_i) \leq H(\pi)$ , with equality if and only if every row of  $Q$  equals  $\pi$ , i.e. consecutive letters are independent.

## Proof

“Conditioning reduces entropy” (Section 8, property 3) with  $X = X_{i+1}$  and  $Y = X_i$ , both of which have pmf  $\pi$  by stationarity:  $H(X_{i+1} | X_i) \leq H(X_{i+1}) = H(\pi)$ , with equality iff  $X_{i+1} \perp\!\!\!\perp X_i$ , i.e.  $Q(x, \cdot) = \pi$  for every  $x$ . □

*A model that uses the previous letter needs no more bits per letter than one that does not, and needs strictly fewer when the previous letter carries information about the next.*

- The saving  $H(\pi) - H(X_{i+1} | X_i) = I(X_i; X_{i+1})$  is the mutual information between consecutive letters.
- The same argument with longer contexts gives  $H(X_{i+1} | X_i, X_{i-1}) \leq H(X_{i+1} | X_i)$ , and so on. Shannon’s table on the slide after next is this chain of inequalities, measured on English.

## Worked Example: Vowels and Consonants

$S = \{V, C\}$ . After a vowel the next letter is a vowel with probability 0.2; after a consonant, 0.5.

### Stationary pmf

Balance the flows  $V \rightarrow C$  and  $C \rightarrow V$ :

$$\pi(V) \cdot 0.8 = \pi(C) \cdot 0.5, \quad \pi(V) + \pi(C) = 1:$$

$$\pi = \left(\frac{5}{13}, \frac{8}{13}\right) = (0.385, 0.615).$$

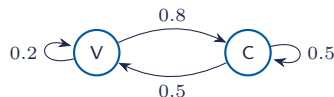
### Entropy rate versus monographic entropy

$$\begin{aligned} H(X_{i+1} | X_i) &= \frac{5}{13} h(0.2) + \frac{8}{13} h(0.5) = \frac{5}{13}(0.722) + \frac{8}{13}(1) \\ &= 0.893 \text{ bits}, \quad H(\pi) = h\left(\frac{5}{13}\right) = 0.961 \text{ bits.} \end{aligned}$$

*Knowing the previous letter saves 0.07 bits per letter here; in English the saving from context is several bits.*

- Digraph check: Markov  $\mathbb{P}(VV) = \pi(V)Q(V, V) = \frac{1}{13} = 0.077$ ; monographic  $\pi(V)^2 = 0.148$ , about twice the Markov value.

| $Q$ | V   | C   |
|-----|-----|-----|
| V   | 0.2 | 0.8 |
| C   | 0.5 | 0.5 |



Row entropies:  $h(0.2) = 0.722$  and  $h(0.5) = 1$ .

# Shannon's Estimates for English

## Bits per letter, 27 symbols (Shannon 1951)

| model           | uniform | monographic | digraphic | trigraphic | long context |
|-----------------|---------|-------------|-----------|------------|--------------|
| bits per letter | 4.75    | 4.03        | 3.32      | 3.1        | about 1      |

*Each richer model of the context lowers the bits needed for the next letter, and with a hundred letters of context a human predictor needs about one bit.*

## The guessing game (the “long context” entry)

Predictor: a person who sees the preceding 100 letters and guesses the next until right. Encoding:  $x_1 \dots x_n \mapsto (g_1, \dots, g_n)$ ,  $g_i$  = number of guesses at position  $i$ . A decoder with the same predictor recovers the text, so

$$H(\text{English}) \leq H(\text{guess counts}) \in [0.6, 1.3] \text{ bits per letter (Shannon's bounds).}$$

- This is the equivalence of prediction and compression (Section 6) with a human predictor, twenty-five years before arithmetic coding.
- Redundancy  $1 - H/\log 27$ : 79% at  $H = 1$ , 73% at  $H = 1.3$ ; Shannon's stated figure is “roughly 75%”.

# Evaluating a Model: Cross-Entropy on Real Text, and Perplexity

## Definition

A model  $q$  gives a pmf  $q(\cdot \mid \text{context}_i)$  for the letter at each position  $i$ . Its **cross-entropy on the text**  $x_1 \dots x_n$  is

$$\frac{1}{n} \sum_{i=1}^n \log \frac{1}{q(x_i \mid \text{context}_i)} \quad \text{bits per letter,}$$

and its **perplexity** is  $\text{PP} = 2^{\text{cross-entropy}} \in [1, \infty)$ .

*The bits per letter the model would need to compress this text; perplexity expresses the same number as an equivalent count of equally likely letters.*

- On long text from a stationary source  $P$  the average tends to the *cross-entropy rate*  $\lim_n H(P_n, q_n)/n$  ( $P_n, q_n$ : pmfs of  $n$ -letter blocks), at least the entropy rate of  $P$  by Gibbs.
- A model giving probability 0 to a letter that occurs has infinite cross-entropy; practical models keep every probability strictly positive.

## Perplexities of Shannon's models

Uniform: 4.75 bits,  $\text{PP} = 27$ . Monographic: 4.03,  $\text{PP} = 16$ . Digraphic: 3.32,  $\text{PP} = 10$ , as if choosing among ten equally likely letters. Trigraphic: 3.1,  $\text{PP} = 8.6$ . A model at 1 bit:  $\text{PP} = 2$ .

## Back to the Cipher: Frequency Analysis

### Theorem (statistics survive a substitution)

Key  $k: S \rightarrow S$  a bijection; ciphertext  $k(X_1), k(X_2), \dots$ . Its monographic and digraphic pmfs are those of the plaintext with the letters relabelled:

$$p_{\text{cipher}}(k(x)) = p(x), \quad p_{2,\text{cipher}}(k(x), k(x')) = p_2(x, x') \quad \text{for all } x, x' \in S.$$

### Proof

$\mathbb{P}(k(X_i) = k(x)) = \mathbb{P}(X_i = x)$  since  $k$  is injective; likewise for pairs. □

*The cipher conceals the identity of each letter but not the frequency of each letter or of each pair.*

- The most frequent ciphertext letter is  $k(\text{E})$ ; the ciphertext letter that is always followed by the same letter is  $k(\text{Q})$ , and what follows it is  $k(\text{U})$ ; the frequent pairs are  $k(\text{T})k(\text{H})$ ,  $k(\text{H})k(\text{E})$ ,  $k(\text{I})k(\text{N})$ ,  $k(\text{E})k(\text{R})$ .
- Entropies survive too:  $H(p_{\text{cipher}}) = H(p)$ , and the entropy rate of the ciphertext is that of English. A substitution preserves the redundancy of the language, and the next slide converts this into a length.

## Back to the Cipher: Unicity Distance

### Unicity distance (Shannon)

Redundancy per letter  $R = \log 26 - H$ ; key entropy  $H(K)$ ; the key is determined after about  $U = H(K)/R$  letters.

| cipher                 | keys   | $H(K)$ bits | $U$ at $H = 1.5$ ( $R = 3.2$ ) | $U$ at $H = 1$ ( $R = 3.7$ ) |
|------------------------|--------|-------------|--------------------------------|------------------------------|
| substitution           | $26!$  | 88.4        | 28 letters                     | 24 letters                   |
| Caesar shift           | 26     | 4.7         | 1.5                            | 1.3                          |
| Vigenère, key length 5 | $26^5$ | 23.5        | 7.3                            | 6.4                          |

*Each ciphertext letter supplies  $R$  bits about the key, so after  $H(K)/R$  letters no key entropy remains.*

- $H = 1.5$  is Shannon's working value from short-context estimates on 26 letters;  $H = 1$  is the long-context figure from Shannon's table of estimates three slides back. The unicity distance is a rough figure either way.
- Below  $U$  several keys give plausible English; above it the decryption is unique. This answers question 3: the length is set by the entropy of the key and the redundancy of the language.